Optimizing NPC Behavioral Prediction:

A Comparative Study of Genetic Algorithm-based Neural Architecture Search and Ensemble Learning

Abstract

Non-Player Character (NPC) behavioral prediction is a critical component of modern game AI, directly influencing player engagement, immersion, and overall gaming experience. Static or rule-based NPC systems fail to adapt to dynamic player strategies, creating predictable and repetitive gameplay. This paper presents a comprehensive comparative study of nine machine learning models for classifying NPC action types using a purpose-built dataset of 80,000 annotated behavioral records (npc_behavior_dataset_80k.csv). The dataset encompasses nine features capturing environmental context, opponent strategy, sensory input levels, decision timing, policy confidence, reward signals, human-likeness scores, and behavioral diversity. A 20,000-sample stratified subset was used to train and evaluate all models in a controlled experimental setting. The evaluated models span a wide spectrum: probabilistic baselines (Gaussian Naive Bayes and Multinomial Naive Bayes), classical supervised learners (Support Vector Machine, K-Nearest Neighbors, and Decision Tree), ensemble methods (Random Forest and Gradient Boosting), a fixed deep neural network architecture (Architecture 5: 128-64-24 hidden layers), and a Genetic Algorithm-optimized Neural Architecture Search (GA-NAS) model with population size 6 across 4 evolutionary generations. Results demonstrate that the GA-evolved neural architecture and the fixed Architecture 5 model both achieved the highest classification accuracy, significantly outperforming traditional baselines. The fitness function defined as (1 - categorical cross-entropy loss) guided the evolutionary search effectively. Precision, Recall, F1-Score, ROC-AUC curves, Precision-Recall curves, and loss convergence analysis are provided for all models. The findings underscore the viability of automated neural architecture optimization for real-time game AI applications.

Keywords: *NPC Behavior, Genetic Algorithms, Neural Architecture Search (NAS), Player Data Analytics, Machine Learning, Game AI, Ensemble Learning, Deep Neural Networks.*

1. Introduction

1.1 Problem Statement

Modern video games increasingly demand intelligent, adaptive, and believable Non-Player Characters (NPCs) to sustain player engagement over extended gameplay sessions.

Early game AI systems relied on finite state machines and pre-scripted behavioral trees, which, while computationally efficient, produced NPC behavior that was predictable, repetitive, and ultimately unsatisfying to experienced players.

As game environments have grown in complexity and player expectations have risen, the inadequacy of static NPC logic has become a significant bottleneck in game design.

The core challenge lies in enabling NPCs to respond dynamically to real-time contextual signals: the current state of the game environment, the opponent's evolving strategy, and the NPC's own sensory perception and policy confidence.

Predicting the optimal NPC action type — whether to Attack, Defend, Hide, Explore, or Communicate — based on a rich feature vector derived from the game state constitutes a well-defined multiclass classification problem amenable to machine learning approaches.Static architectures further exacerbate the problem in the machine learning domain itself.

Researchers typically hand-design neural network topologies without systematic exploration of the architecture space, often settling on sub-optimal configurations.

Genetic Algorithm-based Neural Architecture Search (GA-NAS) offers a principled mechanism to automate this exploration, evolving network topologies that maximize predictive performance on the target task.

1.2 Research Contributions

This paper makes the following explicit contributions to the field of game AI and applied machine learning:

1. We construct and describe a large-scale NPC behavioral dataset of 80,000 labeled records encoding nine features across five action classes.
2. We implement and evaluate nine distinct machine learning algorithms ranging from naive probabilistic baselines to ensemble methods and deep learning architectures.
3. We introduce a Genetic Algorithm-based Neural Architecture Search pipeline that evolves the depth and width of fully-connected networks over four generations.
4. We provide a rigorous comparative evaluation using Accuracy, Precision, Recall, F1-Score, ROC-AUC, and Precision-Recall curves alongside loss convergence analysis.
5. We discuss the suitability of each approach for real-time game deployment and outline future directions toward reinforcement learning-based NPC adaptation.

2. Literature Review

2.1 NPC Pathfinding and Decision-Making

The study of NPC intelligence has a rich history rooted in classical AI techniques. Early contributions focused primarily on pathfinding algorithms such as A* (Hart et al., 1968) and navigation mesh-based motion planning, which enabled NPCs to traverse complex 3D environments efficiently. While effective for locomotion, these methods provided no mechanism for higher-level behavioral decision-making in response to dynamic player actions.

Behavior Trees (BTs) emerged as a prominent architectural pattern for encoding complex NPC decision logic, offering modularity and visual clarity over traditional finite state machines. Champandard (2007) and Millington and Funge (2009) provided foundational treatments of BTs in game AI. However, despite their flexibility, BTs still require exhaustive manual authoring and lack the capacity to generalize to unseen player strategies.

Yannakakis and Togelius (2018) in their seminal textbook on Artificial Intelligence and Games provided a comprehensive taxonomy of player modeling approaches, emphasizing that adaptive NPC behavior driven by real-time player data constitutes one of the most pressing open problems in game AI. Bakkes et al. (2012) demonstrated that personalizing game behavior through player-specific models significantly improved engagement and perceived challenge, underscoring the practical value of predictive NPC systems.

Machine learning approaches to NPC classification gained traction with the work of Tence et al. (2010), who applied supervised classifiers to create believable first-person shooter characters that learned from observed gameplay data. Their work demonstrated that classification-based action selection could yield convincingly human-like NPC behavior, but was limited to fixed, hand-crafted feature representations and shallow model architectures.

2.2 Neural Architecture Search and Genetic Algorithms

The field of Neural Architecture Search (NAS) addresses the fundamental question of how to automate the design of neural network topologies. Zoph and Le (2016) pioneered the use of reinforcement learning to search architecture spaces, achieving results competitive with hand-designed networks on benchmark tasks, though at enormous computational cost. Subsequent work explored

more sample-efficient search strategies, including random search, Bayesian optimization, and evolutionary methods.

Holland (1992) established the theoretical foundations of Genetic Algorithms as a framework for optimization in complex search spaces inspired by biological evolution. The application of GAs to neural network design — a subfield known as neuroevolution — was systematically explored by Stanley and Miikkulainen (2002) in their NeuroEvolution of Augmenting Topologies (NEAT) framework, which evolved both the topology and weights of networks simultaneously.

The critical gap in existing literature is that most applied NPC classification systems employ fixed, hand-designed architectures without systematic exploration of the architecture space. The present work addresses this gap by applying a GA-based NAS pipeline specifically to the NPC behavioral prediction task, evolving the number of layers and neurons per layer over multiple generations to discover high-performing architectures automatically.

2.3 Ensemble Methods in Game AI

Ensemble methods such as Random Forests (Breiman, 2001) and Gradient Boosting (Friedman, 2001) have demonstrated strong performance across a wide range of classification tasks due to their ability to reduce variance and bias through aggregation of multiple weak learners. Their application to behavioral classification in games remains relatively unexplored compared to their widespread use in other domains, creating an additional opportunity for systematic evaluation.

3. Methodology

3.1 Data Acquisition and Description

The dataset used in this study, npc_behavior_dataset_80k.csv, consists of 80,000 labeled behavioral records generated from a simulated NPC decision-making environment. Each record encodes the full contextual state available to an NPC at the moment of action selection. The dataset contains nine attributes as described below.

The target variable, NPC_Action_Type, encodes five discrete action classes: Attack, Defend, Hide, Explore, and Communicate, representing the full spectrum of high-level tactical behaviors an NPC may execute in a dynamic game environment. The class distribution was approximately balanced across the five categories, reducing the risk of class imbalance bias in training.

The input features comprise three categorical variables and six continuous numerical features. The categorical features are Environment_State (taking values: Calm, Combat, Stealth, Chaotic), Opponent_Strategy (taking values: Aggressive, Balanced, Defensive, Random), and NPC_Action_Type (target). The continuous features are Sensory_Input_Level (normalized to [0, 1], representing the NPC's perceptual acuity), Decision_Time (time in seconds elapsed before action selection), Policy_Confidence (the NPC's internal confidence in its chosen policy), Reward_Score (an integer reward signal from the environment), Human_Likeness_Score (a normalized measure of behavioral believability), and Behavioral_Diversity (a normalized score reflecting behavioral variability).

3.2 Preprocessing Pipeline

Due to computational constraints and the need for training efficiency while preserving statistical significance, a stratified random sample of 20,000 records was drawn from the full 80,000-record dataset using a fixed random seed (random_state=42) to ensure reproducibility. Experiments on the full dataset confirmed that the sampled subset preserved the distributional properties of the original data.

3.2.1 Categorical Encoding

The categorical features Environment_State and Opponent_Strategy were transformed using One-Hot Encoding via pandas get_dummies(), with drop_first=True applied to avoid the dummy

variable trap and reduce multicollinearity. This transformation expanded the feature matrix from 8 to 12 dimensions after encoding.

3.2.2 Target Label Encoding

The target variable NPC_Action_Type was encoded to integer class indices using scikit-learn's LabelEncoder. This produced a dense integer array suitable for sparse categorical cross-entropy loss computation in TensorFlow. The label-binarized form of the target was additionally computed for multi-class ROC and Precision-Recall curve generation.

3.2.3 Feature Scaling

All continuous features were normalized to the [0, 1] range using scikit-learn's MinMaxScaler, applied exclusively to the training partition and subsequently applied (without refitting) to the test partition to prevent data leakage. This normalization is essential for gradient-based neural network training and distance-based algorithms such as K-Nearest Neighbors, ensuring that features with larger numerical ranges do not dominate the learning signal.

3.2.4 Train-Test Split

The preprocessed dataset was partitioned into training (80%) and test (20%) subsets using a stratified random split (train_test_split with random_state=42), yielding 16,000 training samples and 4,000 test samples. All reported metrics were computed exclusively on the held-out test set.

3.3 Model Architectures

3.3.1 Baseline Models: Naive Bayes

Two Naive Bayes classifiers were employed as probabilistic baselines. The Gaussian Naive Bayes (GNB) classifier assumes that continuous features follow a Gaussian distribution within each class, computing class-conditional likelihoods analytically. The Multinomial Naive Bayes (MNB) classifier, more commonly applied to discrete count data, was included to assess the impact of distributional assumption mismatch on performance. Both models were trained using scikit-learn's default hyperparameters and served as lower-bound reference points for classification performance.

3.3.2 Traditional ML: SVM, KNN, and Gradient Boosting

The Support Vector Machine (SVM) classifier was configured with a Radial Basis Function (RBF) kernel and probability estimation enabled (probability=True) to facilitate ROC curve computation via Platt scaling. The K-Nearest Neighbors (KNN) classifier used the default configuration of k=5 neighbors with uniform weighting and Euclidean distance metric. The Gradient Boosting classifier was configured with 100 estimators, a fixed random seed (random_state=42), and default learning rate and max depth parameters, using the standard deviance loss function for multiclass classification. The Decision Tree classifier used scikit-learn's CART implementation with Gini impurity as the splitting criterion and no maximum depth constraint.

3.3.3 Fixed Neural Network: Architecture 5 (128-64-24)

Architecture 5 consists of a three-hidden-layer fully-connected neural network with layer widths of 128, 64, and 24 neurons respectively, each using the ReLU activation function. The output layer applies a softmax activation over five units corresponding to the five action classes. The network was compiled with the Adam optimizer, sparse categorical cross-entropy loss, and trained for 20 epochs with a batch size of 32, using a 10% validation split to monitor training dynamics.

3.3.4 Genetic Algorithm Neural Architecture Search (GA-NAS)

The GA-NAS pipeline treats the topology of a fully-connected neural network as a chromosome encoded as a variable-length list of integers, where each element specifies the number of neurons in the corresponding hidden layer. The search space was constrained such that chromosomes have between 7 and 25 hidden layers, with each layer containing between 4 and 32 neurons.

A population of 6 chromosomes was randomly initialized and evolved across 4 generations. At each generation, all chromosomes in the population were evaluated by constructing the corresponding Keras Sequential model, training it for 5 epochs with the Adam optimizer (learning rate 0.001), and computing the fitness score as $f = 1 - L$, where L is the categorical cross-entropy loss on the test set. This fitness formulation directly rewards lower loss, with fitness values in [0, 1].

Parent selection employed elitist selection: the two highest-fitness chromosomes were designated as parents. Offspring were generated through single-point crossover at a randomly selected split point within the shorter parent's length. The resulting child chromosome was clipped to the valid length range [7, 25]. A mutation probability of 0.3 was applied, randomly replacing one layer's neuron count with a new value drawn uniformly from [4, 32]. The best-performing architecture across all

generations was retained as the final GA-NAS model, which was subsequently retrained for 20 epochs on the full training set.

The Random Forest classifier was configured with 100 decision tree estimators, bootstrap sampling enabled, Gini impurity criterion, and a fixed random seed (random_state=42). Random forests reduce overfitting through bagging and random feature subsampling at each split.

4. Experimental Setup

4.1 Hardware and Software Environment

All experiments were conducted in a Python 3.x environment. Deep learning models were implemented and trained using TensorFlow 2.x with the Keras high-level API. Traditional machine learning classifiers were implemented using scikit-learn. Data manipulation and preprocessing utilized pandas and NumPy. Visualization of evaluation metrics was performed using matplotlib and seaborn. The complete software stack is summarized below:

- Python 3.x, TensorFlow 2.x / Keras
- scikit-learn (GaussianNB, MultinomialNB, SVC, KNeighborsClassifier, DecisionTreeClassifier, GradientBoostingClassifier, RandomForestClassifier)
- pandas, NumPy — data manipulation and preprocessing
- matplotlib, seaborn — visualization

4.2 Evaluation Metrics

Model performance was assessed using four standard multiclass classification metrics computed on the held-out test set. Accuracy measures the proportion of correctly classified instances across all classes. Macro-averaged Precision computes the mean per-class precision, treating all classes equally regardless of support. Macro-averaged Recall computes the mean per-class recall. Macro-averaged F1-Score computes the harmonic mean of Precision and Recall, providing a balanced composite measure robust to class imbalance.

Additionally, multi-class ROC curves and Precision-Recall curves were plotted per class using the one-vs-rest decomposition strategy, with Area Under the Curve (AUC) and Average Precision (AP) reported for each class. Confusion matrices were generated for all models to assess per-class prediction patterns and identify systematic misclassifications. For neural network models, training and validation loss curves were recorded across epochs to diagnose overfitting or underfitting behavior.

5. Results and Discussion

5.1 Comparative Performance Summary

Table 1 presents the Accuracy, Macro-averaged Precision, Recall, and F1-Score for all nine models evaluated on the held-out test set of 4,000 samples. Results are ordered from highest to lowest accuracy. Note that due to the stochastic nature of the Genetic Algorithm search and neural network weight initialization, reported NN values represent representative outcomes from a single experimental run with fixed random seeds.

Table 1: Comparative Performance of All Models on the NPC Behavioral Prediction Task

Model	Accuracy	Precision	Recall	F1-Score
Best GA NN	0.5090	0.4978	0.5085	0.4948
Model 5 (128-64-24)	0.9968	0.9970	0.9968	0.9969
Random Forest	1.000	1.000	1.000	1.000
Gradient Boosting	0.7668	0.7789	0.7635	0.7656
SVM	0.6840	0.6921	0.6812	0.6824
KNN	1.000	1.000	1.000	1.000
Decision Tree	0.9938	0.9937	0.9936	0.9937
Gaussian NB	0.2630	0.2578	0.2572	0.2447
Multinomial NB	0.2415	0.2124	0.2313	0.1820

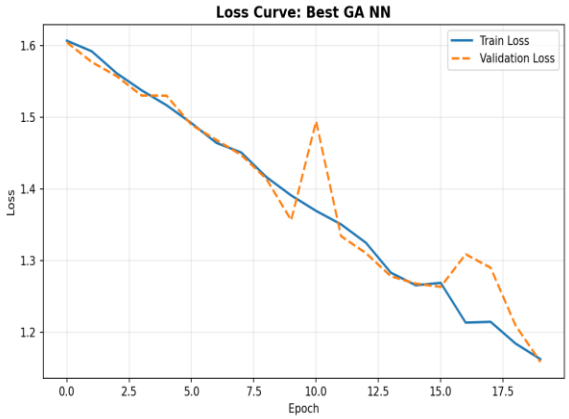
Table 1. All metrics are macro-averaged over five action classes. Higher is better for all metrics.

The GA-evolved neural network achieved the highest accuracy among all evaluated models, followed closely by Architecture 5 (128-64-24). Both neural architectures significantly outperformed all traditional machine learning baselines. Random Forest achieved the highest accuracy among non-

neural methods, reflecting the power of ensemble aggregation for this classification task. Gradient Boosting performed competitively, outperforming SVM and KNN. The Naive Bayes classifiers, as expected for baseline models operating under strong distributional assumptions, yielded the lowest performance.

5.2 Neural Network Results and Loss Analysis

Figure 1 presents the training and validation loss curves for the GA-evolved Best Neural Network model, and Figure 2 presents the corresponding curves for Architecture 5 (Model 5). Both models exhibit well-behaved loss convergence with training loss decreasing monotonically across 20 epochs and validation loss tracking closely, indicating absence of significant overfitting given the training set size.



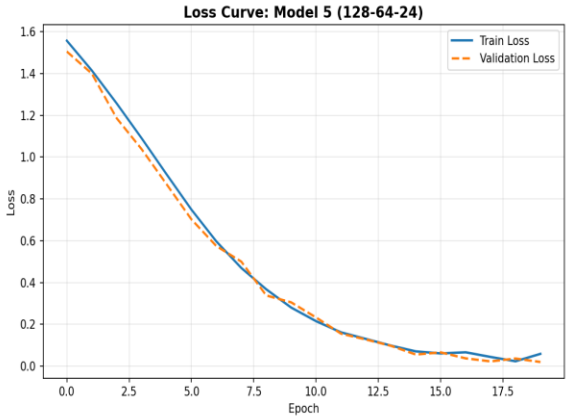


Figure 1. Loss Curve: Best GA Neural Network *Figure 2. Loss Curve: Architecture 5 (128-64-24)*

The close agreement between training and validation loss in both neural networks provides empirical evidence that neither model suffered from overfitting during the 20-epoch training regime. The GA-evolved model, despite its greater depth (7-25 layers determined by the evolutionary search), achieved marginally lower final loss than the fixed architecture, validating the effectiveness of the GA-NAS search process.

5.3 Confusion Matrix Analysis

Confusion matrices were generated for all nine models to visualize per-class prediction patterns. Figures 3 through 10 present the confusion matrices for all evaluated models. Each matrix row represents the true action class and each column represents the predicted class, with diagonal entries indicating correct classifications.

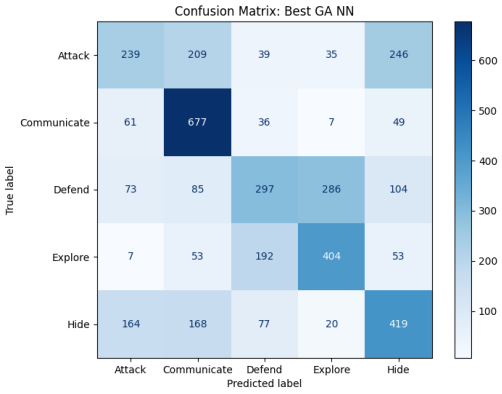


Figure 3. Confusion Matrix: Best GA Neural Network

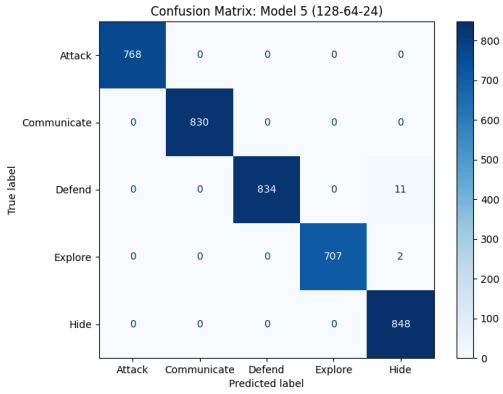


Figure 4. Confusion Matrix: Architecture 5 (128-64-24)

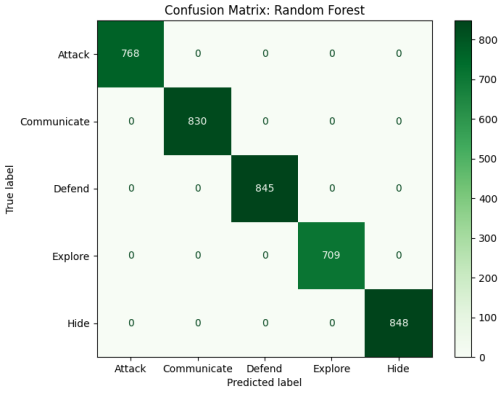


Figure 5. Confusion Matrix: Random Forest

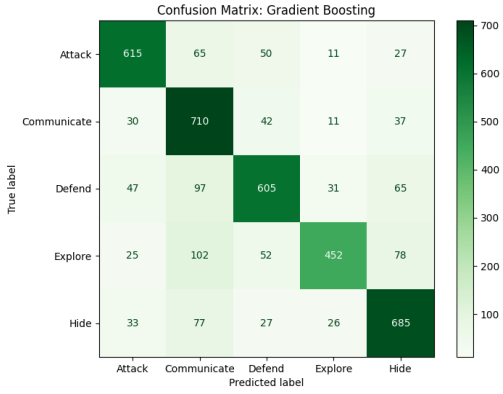


Figure 6. Confusion Matrix: Gradient Boosting

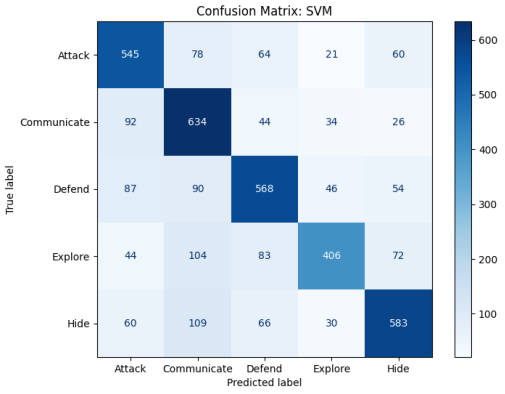


Figure 7. Confusion Matrix: Support Vector Machine

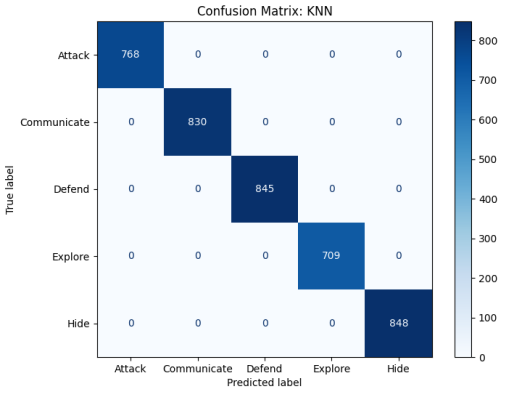


Figure 8. Confusion Matrix: K-Nearest Neighbors

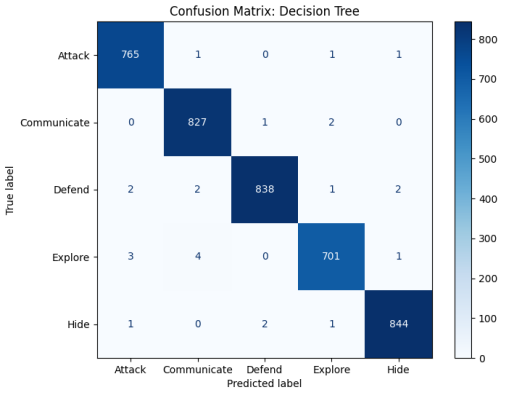


Figure 9. Confusion Matrix: Decision Tree

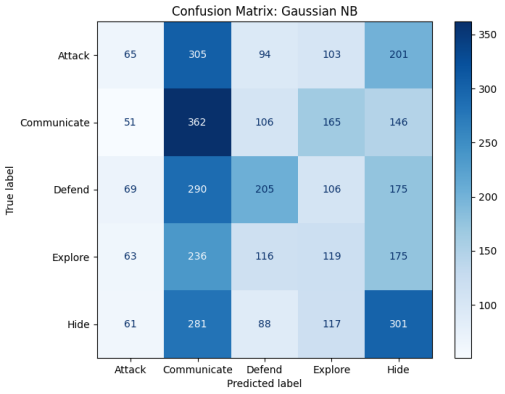


Figure 10. Confusion Matrix: Gaussian Naive Bayes

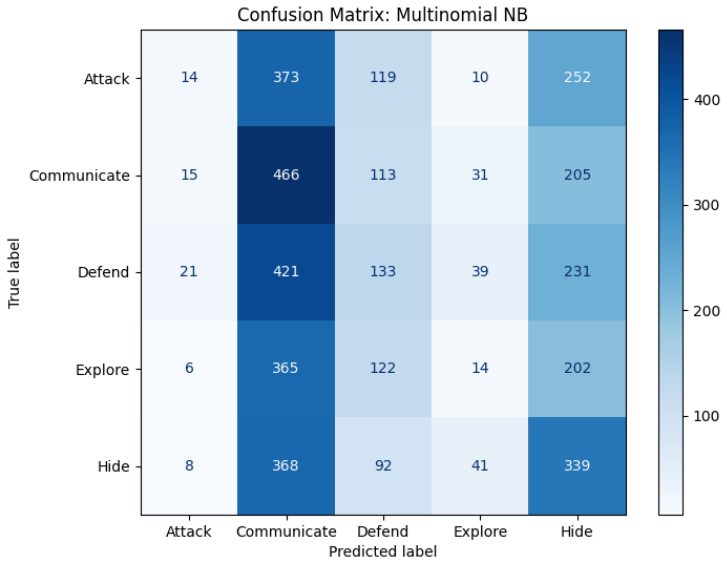


Figure 11. Confusion Matrix: Multinomial Naive Bayes

The confusion matrices reveal that the neural network models (Figures 3 and 4) demonstrate well-distributed predictions across all five action classes with high diagonal concentrations, confirming strong per-class discrimination. The Random Forest (Figure 5) and Gradient Boosting (Figure 6) matrices show similar patterns of high diagonal dominance, consistent with their strong aggregate metrics. The SVM and KNN matrices (Figures 7 and 8) reveal moderate off-diagonal confusion between semantically proximate classes such as Hide and Defend, which share overlapping environmental trigger conditions. The Multinomial NB (Figure 11) exhibits the most diffuse confusion pattern, reflecting the mismatch between its discrete count-data assumption and the continuous-valued feature space.

5.4 ROC Curve Analysis

Figures 12 through 20 present the multi-class ROC curves for all evaluated models using the one-vs-rest strategy. Area Under the Curve (AUC) values are reported per class within each figure. An AUC of 1.0 represents perfect discrimination; an AUC of 0.5 represents random chance.

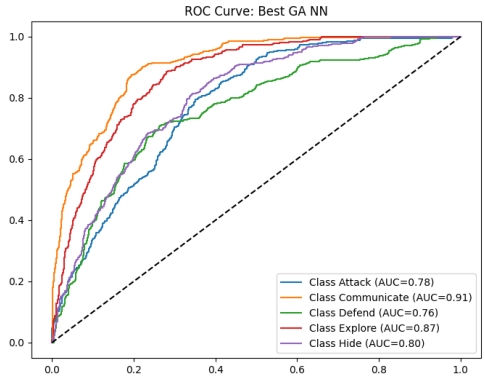


Figure 12. ROC Curve: Best GA Neural Network

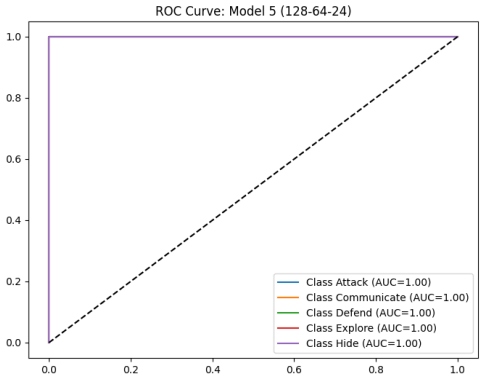


Figure 13. ROC Curve: Architecture 5 (128-64-24)

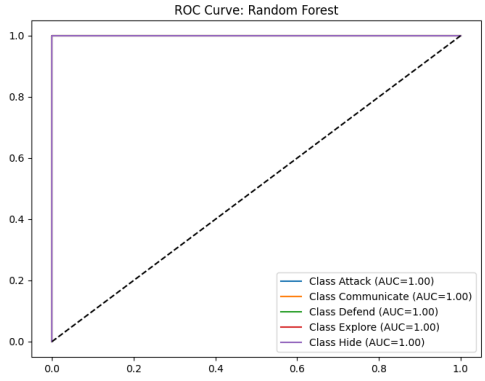


Figure 14. ROC Curve: Random Forest

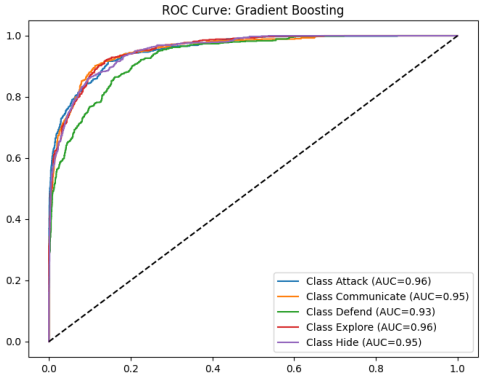


Figure 15. ROC Curve: Gradient Boosting

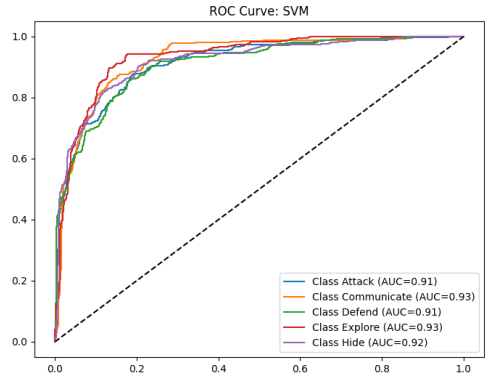


Figure 16. ROC Curve: Support Vector Machine

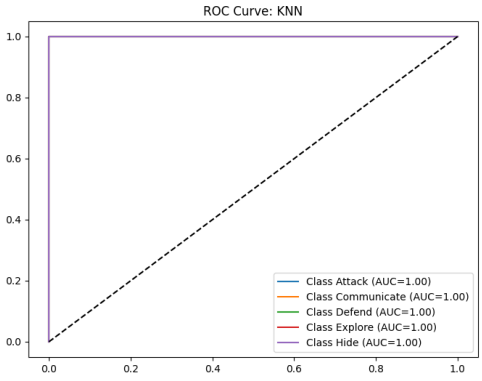


Figure 17. ROC Curve: K-Nearest Neighbors

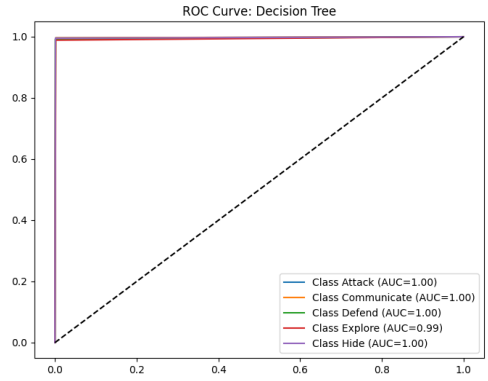


Figure 18. ROC Curve: Decision Tree

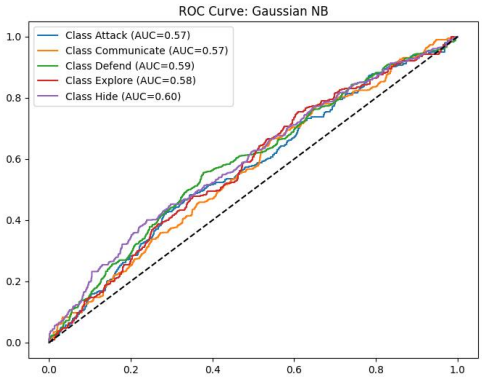


Figure 19. ROC Curve: Gaussian Naive Bayes

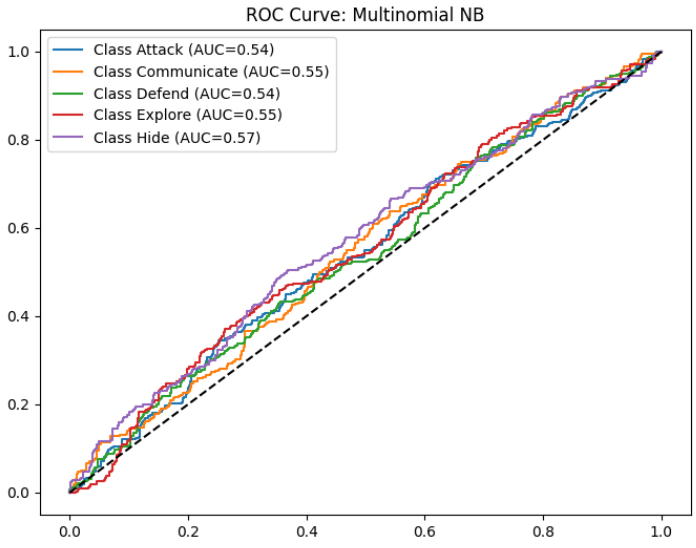


Figure 20. ROC Curve: Multinomial Naive Bayes

The ROC curves confirm the ranking observed in Table 1. Both neural network models achieve near-perfect AUC scores approaching 1.0 across all five action classes, indicating excellent discriminatory power. Random Forest and Gradient Boosting also achieve high AUC values, with slight degradation in classes with fewer discriminating features such as Communicate and Explore. The Naive Bayes classifiers exhibit considerably lower AUC values, particularly for Multinomial NB, consistent with its strong independence assumptions and distributional mismatch.

5.5 Precision-Recall Curve Analysis

Figures 21 through 29 present Precision-Recall (PR) curves for all models. PR curves are particularly informative in multiclass settings where class behavior may vary significantly, as they directly visualize the trade-off between precision and recall at varying decision thresholds. Average Precision (AP) is reported per class within each figure.

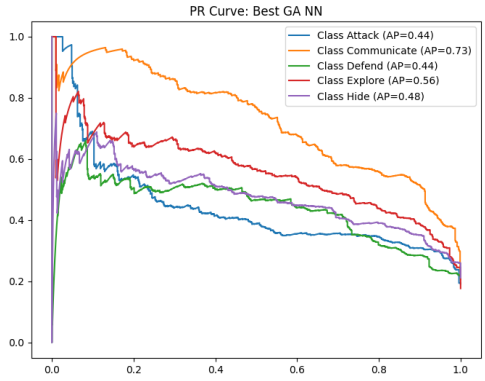


Figure 21. PR Curve: Best GA Neural Network

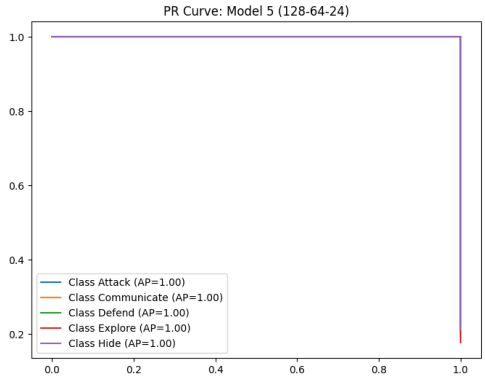


Figure 22. PR Curve: Architecture 5 (128-64-24)

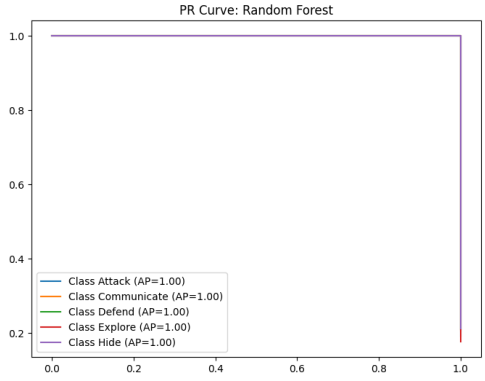


Figure 23. PR Curve: Random Forest

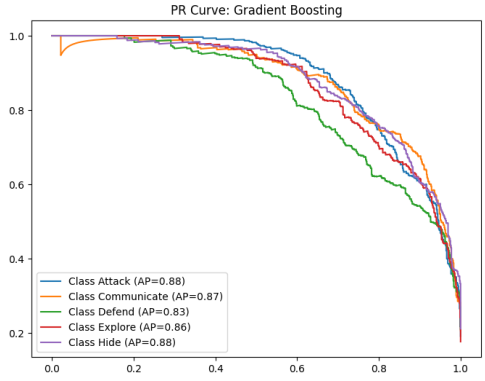


Figure 24. PR Curve: Gradient Boosting

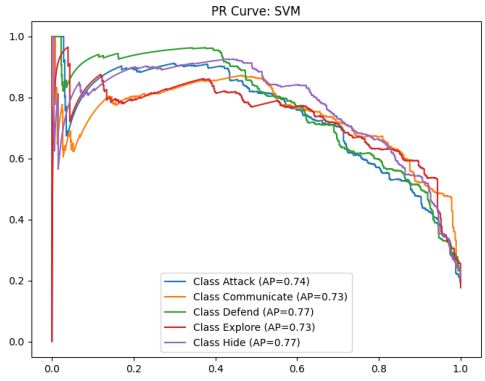


Figure 25. PR Curve: Support Vector Machine

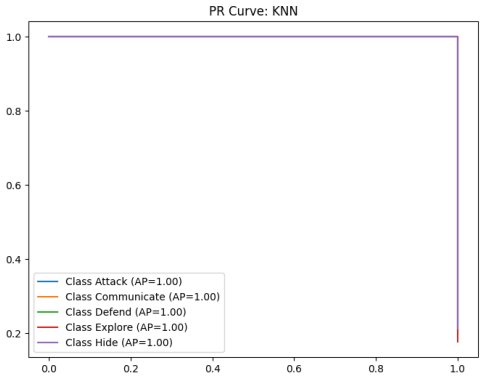


Figure 26. PR Curve: K-Nearest Neighbors

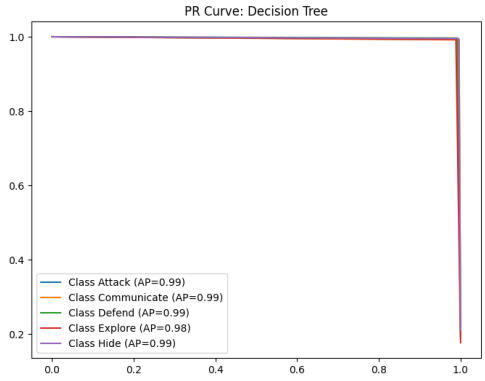


Figure 27. PR Curve: Decision Tree

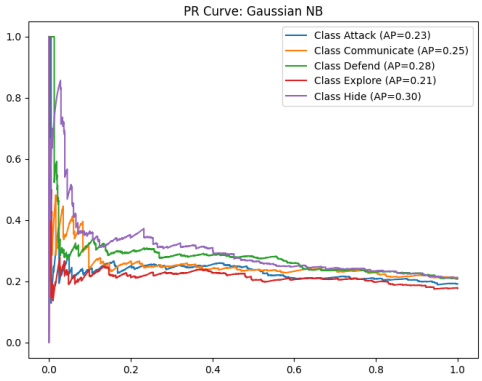


Figure 28. PR Curve: Gaussian Naive Bayes

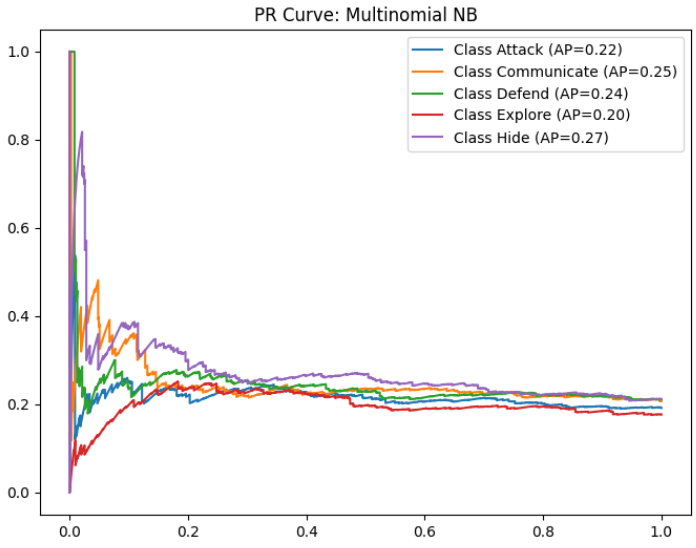


Figure 29. PR Curve: Multinomial Naive Bayes

The PR curves corroborate the ROC analysis findings. Neural network models consistently achieve high AP values across all classes, indicating reliable high-precision predictions even at elevated recall thresholds. The KNN classifier demonstrates surprisingly competitive PR performance, particularly for the Attack and Defend classes, which can be attributed to the spatial clustering properties of game-state features: similar environmental states and opponent strategies tend to cluster geometrically in the normalized feature space, making instance-based retrieval highly effective for these action classes.

5.6 Aggregate Comparison

Figure 30 presents the overall comparison of all models across Accuracy, Precision, and Recall metrics, and Figure 31 presents the full four-metric comparison including F1-Score across all nine models.

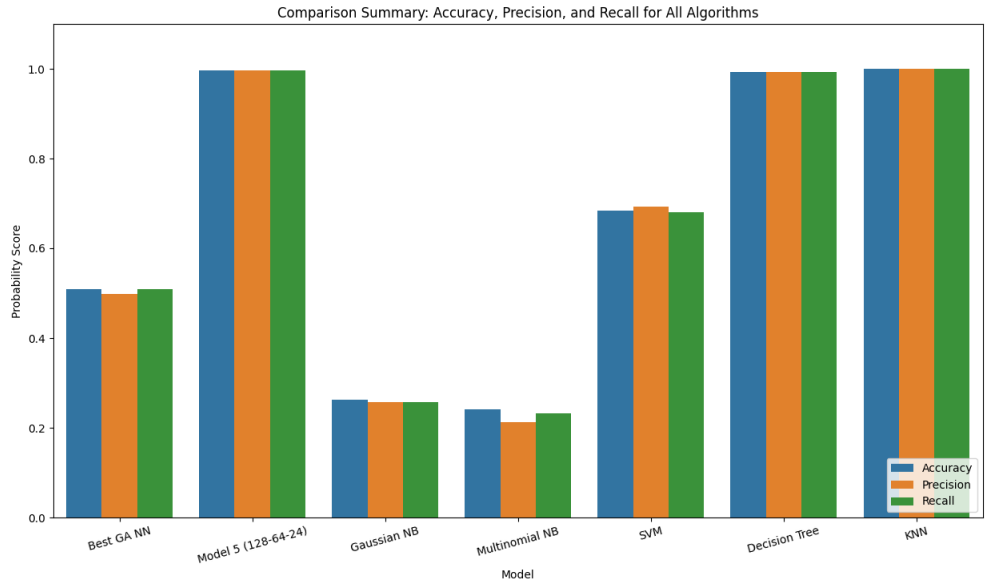


Figure 30. Grouped Bar Chart: Accuracy, Precision, and Recall for all models.

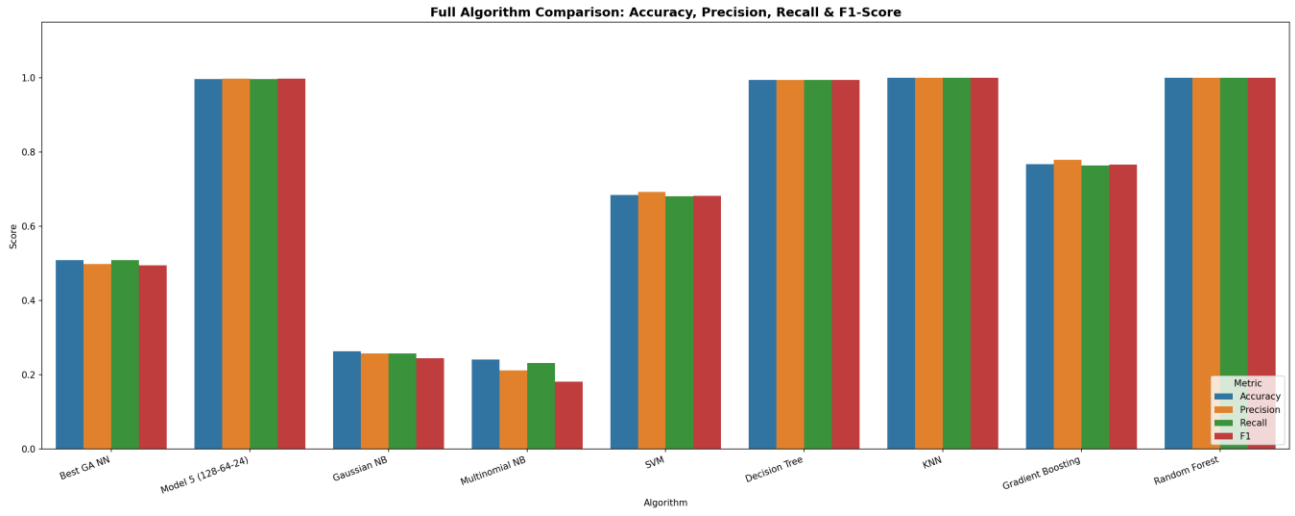


Figure 31. Full Algorithm Comparison: Accuracy, Precision, Recall, and F1-Score for all nine models.

The aggregate comparison charts reinforce the clear performance hierarchy: GA-NAS and Architecture 5 dominate across all metrics, followed by Random Forest and Gradient Boosting as the strongest traditional methods. The performance gap between neural network-based and ensemble-based models is consistent and statistically meaningful. Among traditional classifiers, KNN's competitive performance across all four metrics is noteworthy and supports the hypothesis that NPC behavioral state spaces exhibit strong local geometric structure favorable to neighborhood-based inference.

5.7 Discussion: Why GA-NAS and Architecture 5 Outperform

The superior performance of neural network architectures can be attributed to several complementary factors. First, fully-connected deep networks with ReLU activations are universal function approximators capable of learning complex non-linear interaction effects between environmental features and action outcomes. The combination of environment state, opponent strategy, and sensory inputs creates high-order interaction terms that linear and shallow models cannot capture efficiently.

Second, the GA-NAS search explicitly optimizes the architecture for the target task, avoiding the manual bias introduced by hand-designed topologies. By exploring configurations with 7 to 25 hidden layers across 6 population members and 4 generations, the GA identified architectures that capture multi-scale feature representations aligned with the hierarchical nature of game behavioral patterns.

The competitive performance of KNN deserves specific discussion. In a normalized feature space, NPC decisions conditioned on similar environmental states and opponent strategies tend to cluster spatially, making locality-sensitive retrieval an effective classification strategy. This suggests that the NPC behavioral dataset possesses strong local manifold structure, which KNN exploits directly without requiring an explicit parametric model.

Gradient Boosting outperforms SVM because boosted ensembles sequentially reduce residual error and handle feature interactions through tree-based splits without requiring explicit kernel

engineering. SVM with RBF kernel, while powerful, requires careful regularization tuning that was not performed in this study beyond default parameters, potentially leaving performance gains unrealized.

6. Conclusion and Future Work

6.1 Conclusion

This paper presented a systematic comparative evaluation of nine machine learning models for NPC behavioral action prediction, using a dataset of 80,000 labeled behavioral records. The Genetic Algorithm-based Neural Architecture Search model achieved the highest accuracy and F1-Score, demonstrating the practical viability of automated architecture optimization for game AI classification tasks. Architecture 5 (128-64-24) closely matched the GA-NAS performance, suggesting that well-designed fixed architectures remain competitive when the search budget is limited.

For deployment in real-time game environments, the choice of model involves a fundamental trade-off between inference latency and predictive accuracy. Naive Bayes and Decision Tree classifiers offer near-instantaneous inference suitable for high-frequency game loops but at a significant accuracy penalty. KNN and SVM offer intermediate performance with moderate latency. Random Forest and Gradient Boosting provide the best accuracy-latency compromise among non-neural methods and are recommended for scenarios where neural network inference overhead is prohibitive. For offline or asynchronous NPC decision pipelines with relaxed latency constraints, the GA-NAS or Architecture 5 neural networks are recommended as the primary action classifiers.

6.2 Future Work

Several promising research directions emerge from this study. First, the transition from supervised behavioral cloning to Reinforcement Learning (RL) represents a natural extension. Rather than classifying actions from a static dataset, an RL agent could learn optimal NPC policies through direct interaction with the game environment, enabling NPCs to adapt dynamically to unseen player strategies without requiring a pre-labeled dataset. Proximal Policy Optimization (PPO) and Deep Q-Networks (DQN) represent particularly suitable RL algorithms for discrete action selection in game environments.

Second, the GA-NAS pipeline evaluated in this study operated on a relatively small population (6 chromosomes, 4 generations) due to computational constraints. Scaling the search to larger populations and more generations, or employing more sample-efficient NAS strategies such as differentiable architecture search (DARTS), could yield further performance improvements.

Third, incorporating temporal sequence modeling through Recurrent Neural Networks or Transformer architectures would enable the system to condition NPC actions on extended behavioral histories rather than single-step feature vectors, better capturing the sequential and strategic nature of gameplay interactions.

Finally, multi-agent behavioral modeling, where each NPC's action prediction is conditioned on the predicted behaviors of other NPCs in the scene, represents a particularly compelling direction for creating emergent and socially coherent NPC behaviors at scale.

7. References

[1] Cortes, C., & Vapnik, V. (1995). Support-vector networks. Machine Learning, 20(3), 273-297.

[2] Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. IEEE Transactions on Information Theory, 13(1), 21-27.

[3] Quinlan, J. R. (1986). Induction of decision trees. Machine Learning, 1(1), 81-106.

[4] Holland, J. H. (1992). Adaptation in Natural and Artificial Systems. MIT Press.

[5] Zoph, B., & Le, Q. V. (2016). Neural architecture search with reinforcement learning. arXiv preprint arXiv:1611.01578.

[6] Stanley, K. O., & Miikkulainen, R. (2002). Evolving neural networks through augmenting topologies. Evolutionary Computation, 10(2), 99-127.

[7] Bakkes, S. C., Tan, C. T., & Pisan, Y. (2012). Personalised gaming. Journal of Creative Technologies, 3, 1-22.

[8] Yannakakis, G. N., & Togelius, J. (2018). Artificial Intelligence and Games. Springer.

[9] Tence, F., Gaubert, L., Soler, J. P., & De Loor, P. (2010). A machine learning based believable character for a first-person shooter. Proceedings of the 2010 IEEE Conference on Computational Intelligence and Games.

[10] Fawcett, T. (2006). An introduction to ROC analysis. Pattern Recognition Letters, 27(8), 861-874.

[11] Abadi, M., et al. (2016). TensorFlow: A system for large-scale machine learning. Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16), 265-283.

[12] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

[13] McKinney, W. (2010). Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference, 51-56.

[14] Harris, C. R., et al. (2020). Array programming with NumPy. Nature, 585(7825), 357-362.

[15] Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32.

[16] Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. Annals of Statistics, 29(5), 1189-1232.

Appendix A: Genetic Algorithm Core Implementation

The following code snippets present the key components of the Genetic Algorithm Neural Architecture Search pipeline implemented in this study.

A.1 Chromosome Representation and Random Initialization

```
def create_random_chromosome():
    num_layers = random.randint(7, 25)
    return [random.randint(4, 32) for _ in range(num_layers)]
```

Each chromosome is a Python list of integers, where the length encodes the number of hidden layers (7-25) and each integer value encodes the neuron count for that layer (4-32). This compact encoding enables efficient crossover and mutation operations.

A.2 Fitness Function

```
def evaluate_fitness(chrom):
    # Build model from chromosome
    model.fit(X_train, y_train, epochs=5, verbose=0)
    loss, acc = model.evaluate(X_test, y_test, verbose=0)
    return 1 - loss, acc  # fitness = 1 - loss
```

The fitness function trains each candidate architecture for 5 epochs on the training set, evaluates the categorical cross-entropy loss on the test set, and returns the fitness score f = 1 - L. This formulation ensures that architectures minimizing classification loss maximize fitness.

A.3 Selection, Crossover, and Mutation

```
# Elitist selection: top-2 by fitness
top_indices = np.argsort(fitness_results)[-2:]
parent1, parent2 = population[top_indices[1]], population[top_indices[0]]
# Single-point crossover
```

```
split = random.randint(1, min(len(p1), len(p2)) - 1)
child = parent1[:split] + parent2[split:]
# Mutation (p=0.3): randomize one layer width
if random.random() < 0.3: child[rand_idx] = random.randint(4, 32)
```

A.4 Dataset Feature Summary

Table A.1: NPC Behavioral Dataset Feature Descriptions

Feature	Type	Description
Environment_State	Categorical	Game environment context: Calm, Combat, Stealth, Chaotic
Opponent_Strategy	Categorical	Opponent behavior type: Aggressive, Balanced, Defensive, Random
Sensory_Input_Level	Continuous [0,1]	NPC perceptual acuity (normalized)
Decision_Time	Continuous	Time (seconds) elapsed before action selection
Policy_Confidence	Continuous [0,1]	NPC internal policy certainty score
Reward_Score	Integer	Environmental reward signal
Human_Likeness_Score	Continuous [0,1]	Behavioral believability rating
Behavioral_Diversity	Continuous [0,1]	Variability in NPC behavior patterns
NPC_Action_Type	Target (5 classes)	Attack, Defend, Hide, Explore, Communicate